

# Strategic Warfare Sci-fi UI Starter Pack - Script Documentation



## Introduction

The Strategic Warfare Sci-fi UI Starter Pack is a military-themed user interface designed for strategy, RTS, tower defense, tactical, and base-building games. It includes essential UI components to help you quickly build professional game interfaces while maintaining a consistent visual style.

All assets are provided in high quality and are fully editable, allowing you to customize colors, layouts, and graphics to fit your project.

## Namespace

All scripts are placed under the `Project.UI` namespace to keep the code organized and avoid naming conflicts.

When using these scripts from your own code, add:

```
using Project.UI;
```

## General Requirements

These scripts are designed for Unity Canvas-based UI. Some scripts require Unity UI components such as `Image`, `ScrollRect`, `LayoutGroup`, `LayoutElement`, and `ContentSizeFitter`.

Profile text fields use `TextMeshPro`, so make sure `TextMeshPro` is installed and imported. For hover and drag interactions, the scene should include an `EventSystem`.

---

## ProfileProgressBarUI

### Purpose:

Displays a single progress bar with a name, value text, fill amount, and color state. It is useful for unit stats such as HP, shield, ammo, energy, armor, or build progress.

### How to use:

1. Add the script to a progress bar UI object.
2. Assign the progress name text, value text, fill `RectTransform`, and fill `Image`.
3. Set the high, average, and low progress colors in the Inspector.
4. This script is usually controlled by `ProfileUIPanel` and `ProfileScrollViewSpawner`.

### Main behavior:

- Converts current, minimum, and maximum values into a 0-1 fill amount.
- Scales the fill object on the X axis.

- Changes the fill color based on the progress percentage.
  - Shows values as either current/max or percentage.
- 

## ProfileUIPanel

### Purpose:

Controls one profile card or unit panel. It displays a profile image, name, category, status, and multiple progress bars. It also supports hover color feedback.

### How to use:

1. Add the script to a profile panel prefab.
2. Assign the background image, profile image, TextMeshPro text fields, and progress bar references.
3. Add one or more ProfileProgressBarUI components to the progress bar list.
4. Use this prefab with ProfileScrollViewSpawner to generate profile lists.

### Main behavior:

- Updates profile image, name, category, and status.
  - Updates all assigned progress bars from profile data.
  - Hides unused progress bars when enabled.
  - Changes the panel background color when hovered.
- 

## ProfileScrollViewSpawner

### Purpose:

Automatically creates a scrollable list of profile panels from profile data. It is useful for unit lists, squad lists, building status panels, command rosters, or tactical selection panels.

### How to use:

1. Create a Unity ScrollRect with a content object.
2. Create a profile panel prefab using ProfileUIPanel.
3. Add ProfileScrollViewSpawner to a manager object or the Scroll View object.
4. Assign the ScrollRect, content parent, and profile panel prefab.
5. Add profile entries in the Inspector.
6. Choose vertical or horizontal list direction.
7. Enable Rebuild On Start to generate the list automatically.

### Main behavior:

- Spawns profile panel prefabs into the scroll content.

- Supports vertical or horizontal layouts.
- Adds spacing before and after profile panels.
- Applies optional custom panel width and height.
- Can automatically configure layout groups and content size fitting.
- Includes methods for updating progress values, status text, profile data, list direction, and panel size.

#### Useful methods:

```
SetProfiles(List<ProfileData> newProfiles);
Rebuild();
UpdateProgressValue(int profileIndex, int progressIndex, float newValue);
UpdateProfileStatus(int profileIndex, string newStatus);
UpdateProfileData(int profileIndex, ProfileData newData);
SetListDirection(ListDirection newDirection);
SetGeneralPanelSize(float newWidth, float newHeight);
```

---

## UIDragDropPanel

#### Purpose:

Makes a UI panel draggable inside its parent RectTransform. This is useful for movable command windows, tactical info panels, pop-up menus, floating status panels, or draggable HUD elements.

#### How to use:

1. Add the script to any UI panel with a RectTransform.
2. Leave Drag Handle empty to drag from the whole panel.
3. Assign a header or title bar RectTransform if only part of the panel should be draggable.
4. Enable Keep Inside Parent to keep the panel inside its parent area.
5. Enable Bring To Front On Drag so the dragged panel appears above sibling UI elements.

#### Main behavior:

- Moves the panel based on pointer dragging.
  - Can restrict dragging to a specific handle.
  - Keeps the panel inside parent bounds when enabled.
  - Can ignore layout control to prevent layout groups from fighting the dragged position.
  - Includes SetCanDrag(bool value) to enable or disable dragging from code.
-

## UILoopFadeColor\_NoTransparent

### Purpose:

Creates a looping color fade effect on a UI graphic without changing its transparency. This is useful for alerts, selected states, warning indicators, highlighted buttons, active targets, or sci-fi glowing UI elements.

### How to use:

1. Add the script to a UI object with a Graphic, such as an Image.
2. Assign the target graphic in the Inspector.
3. Choose the start color, end color, and fade duration.
4. Enable `Play On Enable` if the effect should start automatically.
5. Enable `Use Unscaled Time` if the effect should continue while the game is paused.

### Main behavior:

- Loops between two colors using a ping-pong fade.
- Preserves the original alpha when `Keep Original Alpha` is enabled.
- Starts with `Play()` and stops with `StopFade()`.

### Note:

The component class name is `UILoopFadeColor_NoTransparent`. For Unity compatibility, the script file name should match this class name, or the class should be renamed to match the file name.

---

## UIPanelToggleAutoHide

### Purpose:

Shows, hides, and toggles a UI panel, with an optional auto-hide timer. This is useful for temporary command menus, warning panels, quick info windows, tooltips, or notification-style UI panels.

### How to use:

1. Add the script to a button controller object or directly to the panel.
2. Assign the target panel `GameObject`.
3. Connect a UI Button `OnClick` event to `TogglePanel`, `ShowPanel`, or `HidePanel`.
4. Enable `Auto Hide` if the panel should disappear after a set number of seconds.
5. Enable `Bring To Front On Show` if the panel should appear above sibling UI elements.

### Main behavior:

- Toggles the panel active or inactive.

- Moves the panel to the front when shown.
  - Automatically hides the panel after the configured delay.
  - Stops previous hide timers before starting a new one.
- 

## Typical Setup Workflow

1. Create your Canvas and UI layout.
2. Add a Scroll View for profile or unit lists.
3. Create a profile panel prefab using ProfileUIPanel.
4. Add progress bars inside the prefab using ProfileProgressBarUI.
5. Add ProfileScrollViewSpawner to generate the list automatically.
6. Add UIDragDropPanel to floating panels that should be movable.
7. Add UIPanelToggleAutoHide to panels that should open, close, or auto-hide.
8. Add UILoopFadeColor\_NoTransparent to important UI graphics that need a looping highlight effect.

## Customization Tips

- Change colors in the Inspector to match your faction, team, or sci-fi theme.
- Use progress bars for health, shield, ammo, power, morale, or construction progress.
- Use vertical lists for unit rosters and horizontal lists for quick-access command bars.
- Combine draggable panels with auto-hide panels to create flexible tactical UI windows.
- Use looping color fades for warnings, active selections, alerts, or mission-critical indicators.